Basic R Implementation Vignette

Team Chou Why Did: Renfei Wu, Shiqi Tian, Chengxi Li, Liwen Yin

0. Introduction

The project contain 4 R files:

1. `data_simulation.R`
2. `permutation_test.R`
3. `bootstrap_stability.R`
4. `evaluation.R`

To run the entire project, execute each class in the order described above. The first R script is used to generate sample data. The second and third R scripts allow users to choose hyperparameters and perform statistical tests to identify important features. The final script evaluates the accuracy of their self-defined statistical method, for example by measuring how successfully it identifies the correct features.

1. Data Simulation

(1) `simulate_mixture_data()`

This function generates a synthetic dataset for testing feature selection methods. It creates an $n \times p$ feature matrix X and binary labels Y, where only `n_relevant` features are truly associated with Y (the rest are pure noise). The indices of the true relevant features are stored in S, which serves as ground truth for evaluation later.

```
sim <- simulate_mixture_data(
  n          = 200,
  p          = 50,
  n_relevant = 10,
  alpha      = 0.7,
  dist_G     = "normal",
  dist_H     = "normal",
  seed       = 380
)
```

The upward example create a dataset which has 200 observations and 50 features, of which 10 are truly relevant. The signal strength is 10. The signal and noise distribution are all normal.

Notice, the inputs has the following constrain:

- For `alpha`, it should be in range $[0, 1]$
- For `dist_G`, the valid inputs are
  - "normal": mean shift by class
  - "normal_scale": mean and scale differ by class

- For `dist_H`, the valid inputs are
  - "normal"
  - "uniform"

These indices in `sim$S` will be used as ground truth throughout the rest of the examples shown in this document.

Here we will show some information for `sim`:

```
names(sim)  # Class property
#> [1] "X"      "Y"      "S"      "theta"  "phi"    "alpha"  "dist_G" "dist_H"
```

The detail descriptions for each property can be found in the corresponding R file's function header.

```
dim(sim$X)  # Dimension
#> [1] 200  50
sort(sim$S)  # True relevant feature indices
#>  [1] 13 17 18 20 23 30 33 40 42 49
table(sim$Y)  # Data for each class
#>
#>   0   1
#> 100 100
```

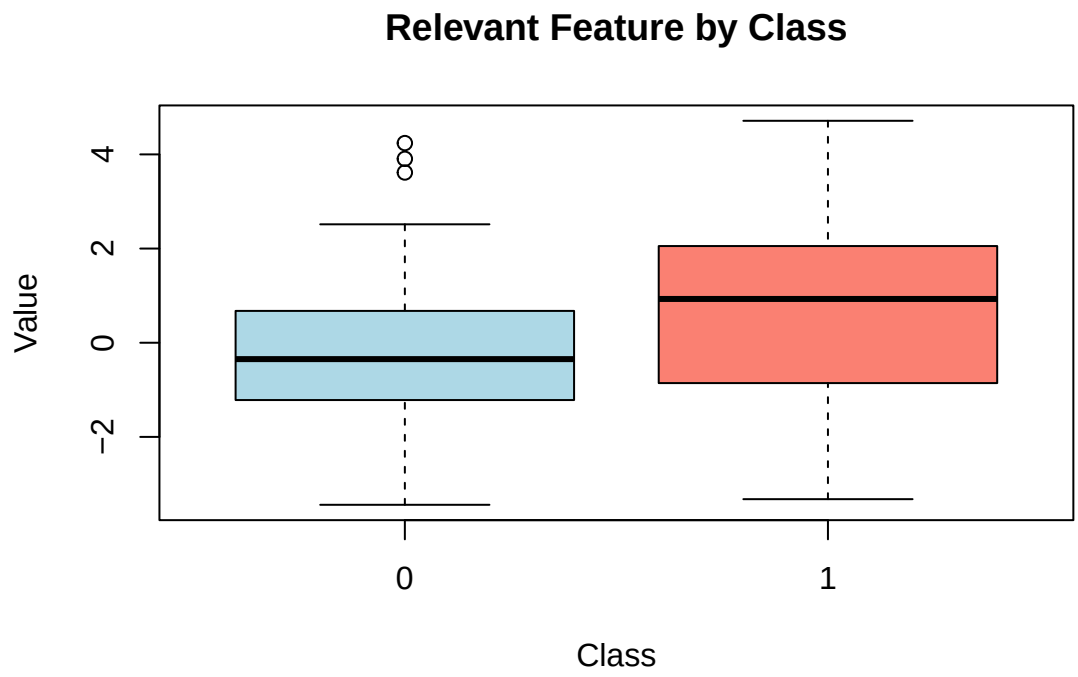
2. Permutation Testing

(1) `stat_mean_diff()`

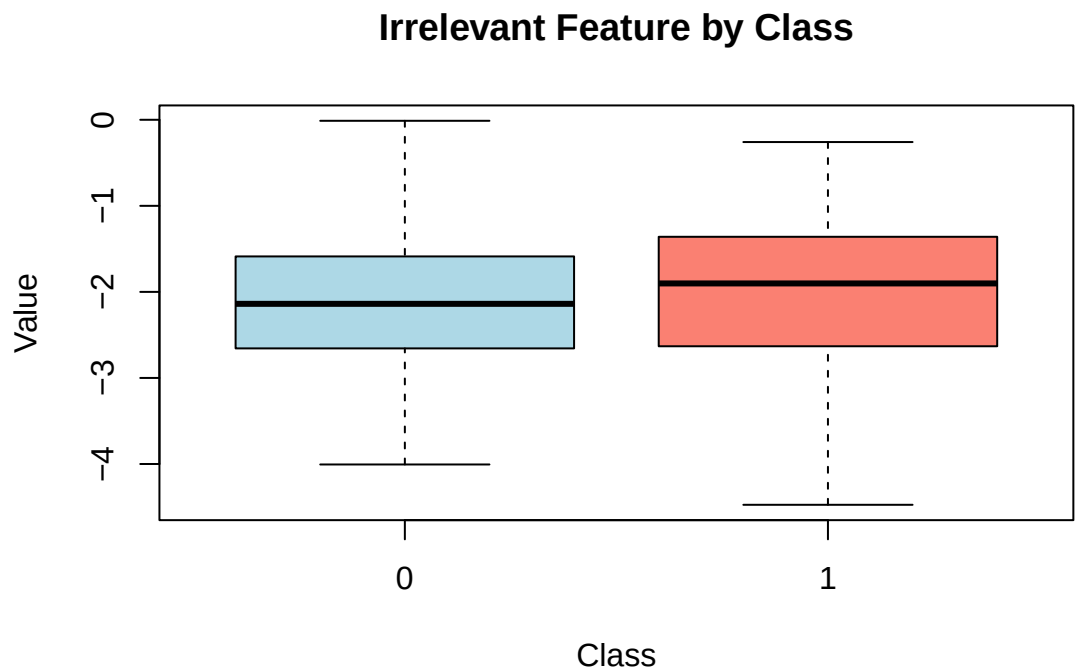
This function computes the absolute difference between the group means of a single feature. A larger value indicates a stronger difference between the two classes. Which is:

$$\text{stat} = |\bar{x}_1 - \bar{x}_0|$$

```
# Get the most relevant feature
stats <- sapply(sim$S, function(j) stat_mean_diff(sim$X[, j], sim$Y))
best_j <- sim$S[which.max(stats)]

x_relevant <- sim$X[, best_j]  # most strongly relevant feature
x_irrelevant <- sim$X[, setdiff(1:50, sim$S)[1]] # others

stat_mean_diff(x_relevant, sim$Y)
#> [1] 0.8725607
stat_mean_diff(x_irrelevant, sim$Y)
#> [1] 0.103858

# display plot
boxplot(x_relevant ~ sim$Y,
        main = "Relevant Feature by Class",
        xlab = "Class",
        ylab = "Value",
        col = c("lightblue", "salmon"))
```




```r
boxplot(x_irrelevant ~ sim$Y,
        main = "Irrelevant Feature by Class",
        xlab = "Class",
        ylab = "Value",
        col = c("lightblue", "salmon"))
```



The relevant feature produces a larger statistic than the irrelevant one, confirming that the function captures the signal correctly.

The boxplots show that the relevant feature exhibits a clear separation between the two classes, whereas the irrelevant feature shows substantial overlap. This visual difference aligns with the larger test statistic observed earlier.

(2) `stat_ks()`

This function computes the Kolmogorov-Smirnov statistic, which measures the maximum difference between the empirical distributions of a feature in class 0 and class 1. Unlike `stat_mean_diff()`, it captures differences in the overall shape of the distribution, not just the mean.

Notice, the inputs has the following constrain:

- `x` should be a numeric vector (a single feature).
- `Y` should be binary class labels (0/1) with both classes present.

```
stat_ks(x_relevant, sim$Y)
#> [1] 0.31
stat_ks(x_irrelevant, sim$Y)
#> [1] 0.14
```

Again, the relevant feature produces a larger statistic, indicating a greater distributional difference between the two classes.

(3) `stat_cvm()`

This function computes the Cramér-von Mises statistic, which measures the overall difference between two empirical distributions by summing the squared differences at all observed points. It is more sensitive to differences across the whole distribution compared to `stat_ks()` (Good 2005).

Notice, the inputs has the following constrain:

- `x` should be a numeric vector (a single feature).
- `Y` should be binary class labels (0/1) with both classes present.

```
stat_cvm(x_relevant, sim$Y)
#> [1] 1.5276
stat_cvm(x_irrelevant, sim$Y)
#> [1] 0.2581
```

As expected, the relevant feature yields a larger statistic than the irrelevant feature.

(4) `permutation_pvalue_one()`

This function computes a permutation p-value for a single feature. It repeatedly shuffles the class labels $\mathbf{Y}$ and recomputes the test statistic each time, building a null distribution. The p-value is the proportion of shuffles that produced a statistic as large as the observed one. A small p-value suggests the feature is truly associated with $\mathbf{Y}$.

Notice, the inputs has the following constrain:

- `stat_fun` must be a function of the form `(x, Y) -> scalar` (e.g. `stat_mean_diff`).
- `n_shuffles` controls Monte Carlo accuracy; larger values are slower but more stable.

```
set.seed(380)
permutation_pvalue_one(x_relevant, sim$Y,
                       stat_fun   = stat_mean_diff,
                       n_shuffles = 199)
#> [1] 0.005

permutation_pvalue_one(x_irrelevant, sim$Y,
                       stat_fun   = stat_mean_diff,
                       n_shuffles = 199)
#> [1] 0.27
```

The relevant feature gives a much smaller p-value than the irrelevant feature, indicating strong evidence of association with $\mathbf{Y}$.

(5) `permutation_pvalues()`

This function applies `permutation_pvalue_one()` to every column of $\mathbf{X}$, returning a vector of p-values of length p. This allows us to assess all features at once.

Notice, the inputs has the following constrain:

- `X` should be an $n \times p$ numeric matrix.

- `Y` should be length n with values 0/1.

```
set.seed(380)
pvals <- permutation_pvalues(sim$X, sim$Y,
                             stat_fun = stat_mean_diff,
                             n_shuffles = 199)
length(pvals)
#> [1] 50
summary(pvals)
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>  0.0050  0.1812  0.4125  0.4249  0.6200  0.9450
```

The function returns one p-value per feature. Features with small p-values are candidates for selection.

(6) `select_features_permutation()`

This function selects features whose permutation p-value is at or below a chosen significance level `alpha_level`. It is the final step of the permutation testing pipeline.

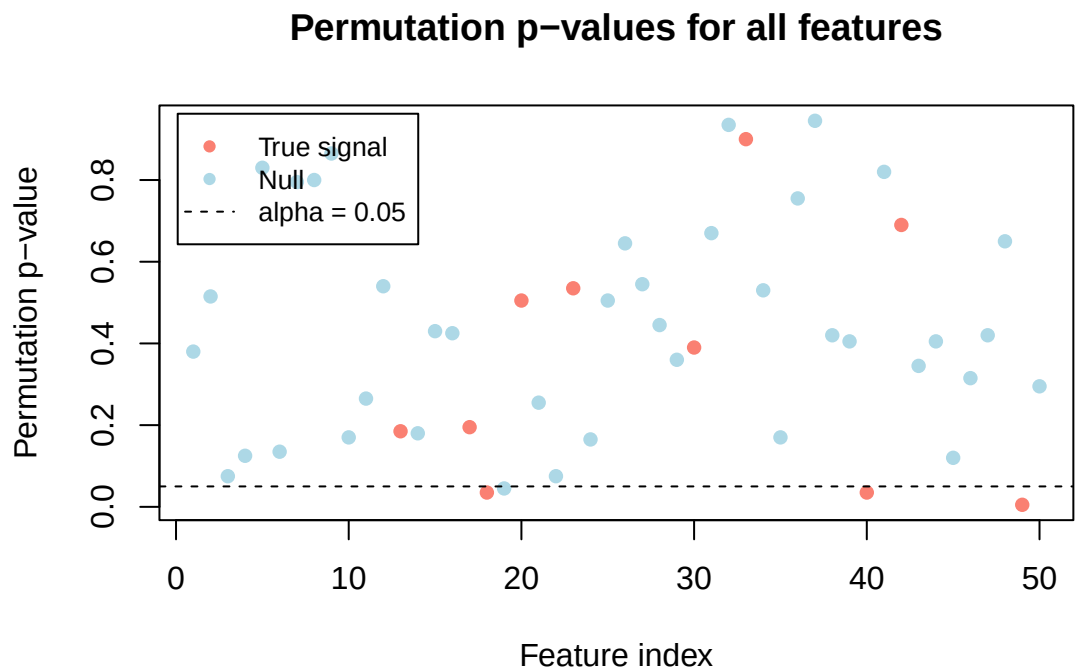
Notice, the inputs has the following constrain:

- `alpha_level` should be in range $(0, 1)$, typical values are 0.05 or 0.01.

```
set.seed(380)
selected_perm <- select_features_permutation(
  sim$X, sim$Y,
  stat_fun    = stat_mean_diff,
  n_shuffles  = 199,
  alpha_level = 0.05
)

selected_perm  # selected feature indices
#> [1] 18 19 40 49
sim$S          # true relevant features for comparison
#>  [1] 13 17 18 20 23 30 33 40 42 49
```

display the p-values for all 50 features:

```
plot(pvals,
     xlab = "Feature index",
     ylab = "Permutation p-value",
     main = "Permutation p-values for all features",
     pch  = 16,
     col  = ifelse(1:50 %in% sim$S, "salmon", "lightblue"))
abline(h = 0.05, lty = 2)
legend("topleft",
       legend = c("True signal", "Null", "alpha = 0.05"),
       col    = c("salmon", "lightblue", "black"),
       pch    = c(16, 16, NA),
       lty    = c(NA, NA, 2),
       inset = 0.02,
       cex = 0.85)
```



Features below the dashed line are selected. True signal features are shown in red.

3. Bootstrap Stability

(1) `bootstrap_selection_frequency()`

A single permutation test can vary depending on the sample. This function runs the feature selection procedure across B bootstrap resamples and records how often each feature is selected. A higher frequency indicates a more stable and reliable feature (Efron and Tibshirani 1994).

```
selector <- function(X, Y) {
  select_features_permutation(X, Y,
                              stat_fun    = stat_mean_diff,
                              n_shuffles  = 99,
                              alpha_level = 0.05)
}

set.seed(380)
freq <- bootstrap_selection_frequency(
  sim$X, sim$Y,
  select_fun = selector,
  B          = 50,
  seed       = 380
)

length(freq)
#> [1] 50
round(freq, 2)
```

```
#>  [1] 0.16 0.18 0.32 0.44 0.08 0.38 0.08 0.00 0.00 0.28 0.10 0.02 0.22 0.26 0.14
#> [16] 0.04 0.16 0.68 0.46 0.10 0.24 0.58 0.08 0.28 0.18 0.04 0.08 0.12 0.18 0.08
#> [31] 0.04 0.02 0.04 0.10 0.38 0.16 0.02 0.10 0.16 0.62 0.08 0.10 0.08 0.12 0.22
#> [46] 0.10 0.06 0.04 0.92 0.20
```

Features in the true signal set tend to have higher selection frequencies than noise features.

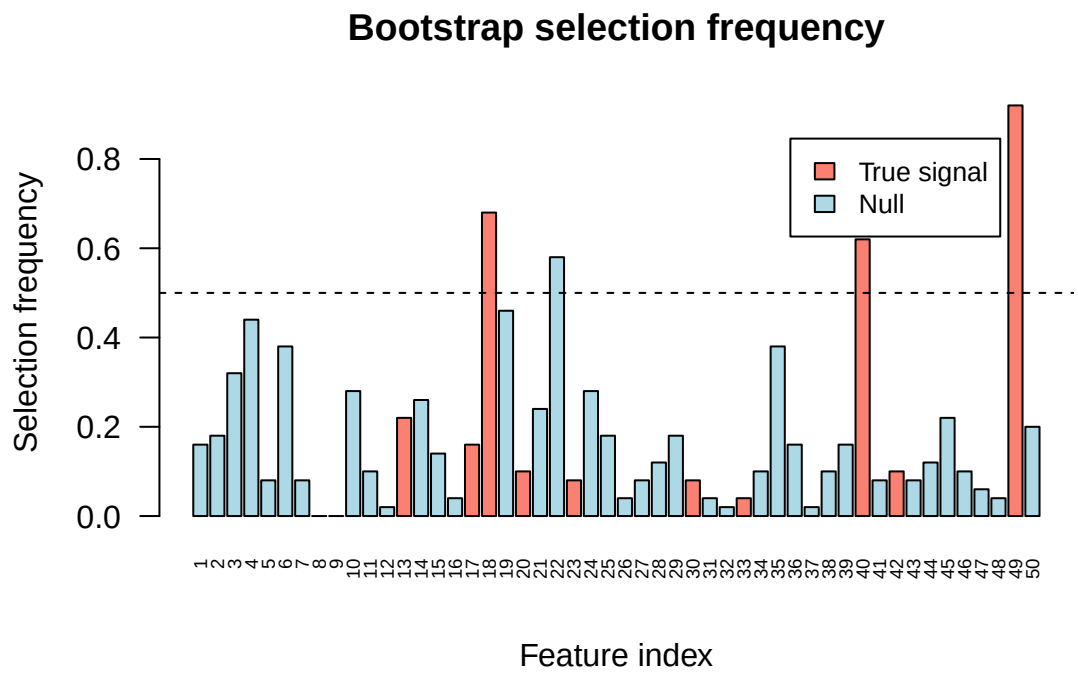
(2) `select_features_bootstrap()`

This function selects features whose bootstrap selection frequency is at or above a `threshold`. Only features that are consistently selected across bootstrap resamples are retained, making the selection more stable. It also returns the full frequency vector as an attribute.

Notice, the inputs has the following constrain:

- `threshold` should be in range $[0, 1]$. Larger values mean stricter selection.
- `B` trades off stability vs runtime (more bootstraps = slower, smoother frequencies).

```r
set.seed(380)
selected_boot <- select_features_bootstrap(
  sim$X, sim$Y,
  select_fun = selector,
  B          = 50,
  threshold  = 0.5,
  seed       = 380
)

selected_boot
#> [1] 18 22 40 49
#> attr(,"frequency")
#>  [1] 0.16 0.18 0.32 0.44 0.08 0.38 0.08 0.00 0.00 0.28 0.10 0.02 0.22 0.26 0.14
#> [16] 0.04 0.16 0.68 0.46 0.10 0.24 0.58 0.08 0.28 0.18 0.04 0.08 0.12 0.18 0.08
#> [31] 0.04 0.02 0.04 0.10 0.38 0.16 0.02 0.10 0.16 0.62 0.08 0.10 0.08 0.12 0.22
#> [46] 0.10 0.06 0.04 0.92 0.20
attr(selected_boot, "frequency")[selected_boot]
#> [1] 0.68 0.58 0.62 0.92

barplot(attr(selected_boot, "frequency"),
        names.arg = 1:50,
        col       = ifelse(1:50 %in% sim$S, "salmon", "lightblue"),
        xlab      = "Feature index",
        ylab      = "Selection frequency",
        main      = "Bootstrap selection frequency",
        las       = 2,
        cex.names = 0.6)
abline(h = 0.5, lty = 2)
legend("topright",
       legend = c("True signal", "Null"),
       fill   = c("salmon", "lightblue"),
       inset   = 0.08,
       cex     = 0.85)
```



4 features above the dashed line are selected. True signal features are shown in red.

4. Evaluation

(1) `evaluate_precision_recall()`

This function compares the selected features against the true relevant set **S**. Precision measures how many selected features are truly relevant. Recall measures how many true relevant features were found.

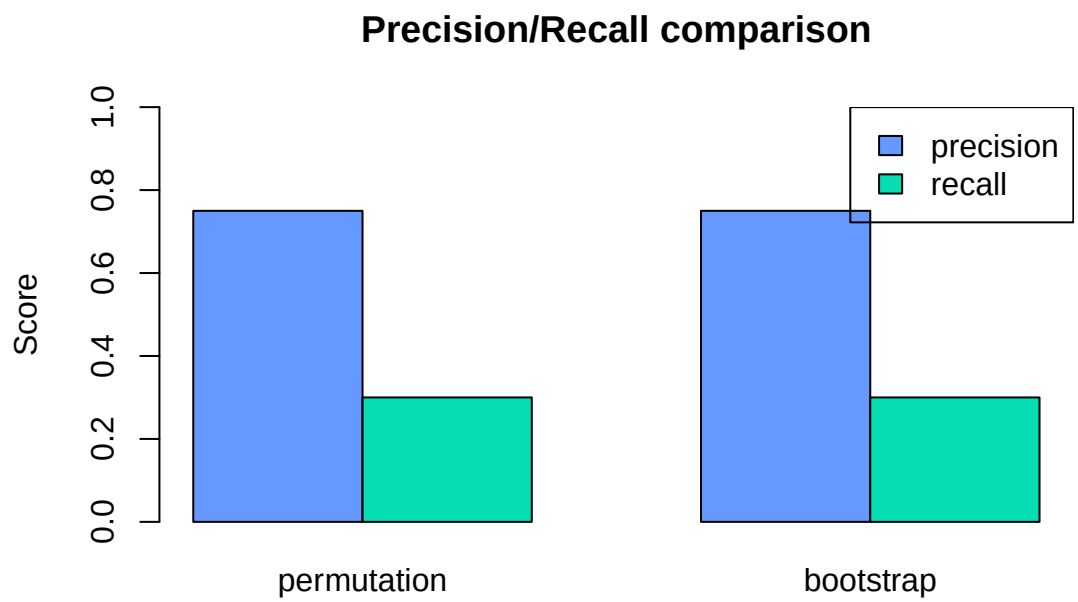
In this section, we evaluate both the direct permutation selection and the bootstrap-stabilized selection, then visualize the trade-off.

```
pr_perm <- evaluate_precision_recall(S_truth = sim$S, S_hat = selected_perm)
pr_boot <- evaluate_precision_recall(S_truth = sim$S, S_hat = selected_boot)

pr_perm$precision
#> [1] 0.75
pr_perm$recall
#> [1] 0.3
pr_boot$precision
#> [1] 0.75
pr_boot$recall
#> [1] 0.3
```

A higher precision means fewer false positives were selected. A higher recall means more true relevant features were found.

We can also compare both methods in a single plot:

```r
mat <- rbind(
  permutation = c(precision = pr_perm$precision, recall = pr_perm$recall),
  bootstrap   = c(precision = pr_boot$precision, recall = pr_boot$recall)
)
barplot(
  t(mat),
  beside = TRUE,
  col = c("#6699FF", "#05DEB2"),
  ylim = c(0, 1),
  ylab = "Score",
  main = "Precision/Recall comparison"
)
legend("topright", legend = colnames(mat), fill = c("#6699FF", "#05DEB2"))
```



(2) `f1_score()`

The F1 score combines precision and recall into a single number. Both must be high for a good F1 score (Voss 2013).

$$F1 = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

```r
f1_perm <- f1_score(pr_perm$precision, pr_perm$recall)
f1_boot <- f1_score(pr_boot$precision, pr_boot$recall)
f1_perm
#> [1] 0.4285714
f1_boot
#> [1] 0.4285714
```

An F1 score closer to 1 indicates better overall feature selection performance.

Generative AI Statement

ChatGPT was used in this project to assist with two aspects:

- Generating the plotting code for the visualizations in the vignette, and
- Find any grammar issue for the documentation.

All code and descriptions were reviewed and verified by the author.

Bibliography

Chang, Winston, Joe Cheng, JJ Allaire, et al. 2026. *Shiny: Web Application Framework for r*. https://shiny.posit.co/.

Efron, Bradley, and Robert J Tibshirani. 1994. *An Introduction to the Bootstrap*. Chapman; Hall/CRC.

Good, Phillip. 2005. *Permutation, Parametric and Bootstrap Tests of Hypotheses*. Springer.

R Core Team. 2025. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. https://www.R-project.org/.

Voss, Jochen. 2013. *An Introduction to Statistical Computing: A Simulation-Based Approach*. John Wiley & Sons.